SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

CO1 ASSESSMENT TOOL 1 – Case Study

Course Code:	CSA10 / CSA1004	**Course Title:**	Software Engineering
CO Assessed:	CO1	**Assessment:**	Assessment Tool 1 – **Case Study**
Weightage:	20%	**Total Marks:**	25
CO1 Statement:	Apply advanced methodologies and agile project management techniques to manage software projects effectively		
Student Name:	R.Nandhini	**Reg. No.:**	192524355
Date:		**Duration:**	60 minutes

Code of Conduct

I R.Nandhini, 192524355 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

Annexure A – Agile Sprint Planning & User Story Workshop

Instructions to Students:

Each student is assigned an individual software project problem statement. Based on the given problem statement, perform an **Agile Sprint Planning and User Story Workshop** by applying Agile software engineering practices. Prepare a comprehensive sprint planning document covering the following aspects:

Based on the given problem statement, perform an **Agile Sprint Planning and User Story Workshop** by applying Agile software engineering practices. Prepare a comprehensive sprint planning document covering the following aspects:

1. **Identify the Project Vision and Stakeholders**
   - Define the project's vision, objectives, and identify all key stakeholders involved.
2. **Create Epics and User Stories**
   - Break down the requirements into Epics and formulate well-structured User Stories using the format:

   As a <user>, I want <feature>, so that <benefit>.
3. **Define Acceptance Criteria**
   - Write clear and measurable acceptance criteria for each selected user story using the Given–When–Then format or equivalent.
4. **Prioritize the Product Backlog**
   - Organize and prioritize the product backlog based on business value, customer needs, and project dependencies.
5. **Plan the Sprint Backlog**
   - Select high-priority user stories for the first sprint and divide them into actionable development tasks.
6. **Estimate Effort Using Story Points**
   - Assign story points to each selected user story using an appropriate estimation technique (e.g., Fibonacci sequence or Planning Poker) and justify your estimates.
7. **Present the Sprint Goal and Expected Deliverables**
   - Define a clear sprint goal and describe the expected sprint deliverables, including the features to be completed at the end of the sprint.

Rubrics for Calculation

Criteria	Excellent (5 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement (1 Mark)
Project Vision & Stakeholder Identification (30%)	Project vision is clear, comprehensive, and aligned with the problem statement. All relevant stakeholders and their roles are correctly identified.	Vision is clear with minor omissions. Most stakeholders are correctly identified.	Vision is partially defined. Some stakeholders are missing or incorrectly identified.	Vision is unclear or incomplete. Stakeholders are poorly identified or missing.
Epics & User Stories (25%)	Epics are well-defined and user stories are complete, follow Agile format, and fully capture functional requirements.	Epics and user stories are mostly complete with minor formatting or requirement issues.	User stories are partially complete with limited Agile structure.	User stories are incomplete, unclear, or do not follow Agile practices.
Acceptance Criteria (20%)	Acceptance criteria are specific, measurable, testable, and aligned with all user stories.	Acceptance criteria are mostly clear and testable with minor gaps.	Acceptance criteria are partially defined and lack clarity or completeness.	Acceptance criteria are missing or not aligned with user stories.
Product Backlog Prioritization & Sprint Planning (15%)	Product backlog is logically prioritized, sprint backlog is realistic, and task breakdown is complete.	Backlog prioritization and sprint planning are mostly appropriate with minor issues.	Prioritization or sprint planning is partially complete with limited justification.	Backlog prioritization and sprint planning are poorly organized or incomplete.
Story Point Estimation, Sprint Goal & Presentation (10%)	Story point estimation is well justified, sprint goal is clear, deliverables are realistic, and presentation is professional and well organized.	Estimation and sprint goal are mostly appropriate with minor inconsistencies. Presentation is clear.	Estimation or sprint goal lacks justification. Presentation is adequate but incomplete.	Estimation is unrealistic or missing. Sprint goal, deliverables, and presentation are unclear or incomplete.

Criteria	**Mark**
Project Vision & Stakeholder Identification (30%)	/5
Epics & User Stories (25%)	/5
Acceptance Criteria (20%)	/5
Product Backlog Prioritization & Sprint Planning (15%)	/5
Story Point Estimation, Sprint Goal & Presentation (10%)	/5
TOTAL	/25

INTELLIGENT VEHICLE MAINTENANCE MANAGEMENT SYSTEM

Project Management using a Kanban Board

1. AIM

To create and organize a Jira project for the Intelligent Vehicle Maintenance Management System by adding development tasks and tracking their progress using a Kanban board.

2. INTRODUCTION

Organizations that operate cars, buses, trucks, vans, and other vehicles must maintain them regularly. Servicing helps vehicles remain safe, dependable, and ready for use. When service dates are missed or faults are not recorded, vehicles may break down, repairs may become expensive, and daily operations may be delayed.

The Intelligent Vehicle Maintenance Management System is a proposed system that organizes vehicle details, service histories, maintenance schedules, faults, reminders, spare-parts usage, repair costs, and vehicle utilization. Jira is used in this practical exercise to plan and monitor the tasks needed to develop the system.

This report analyses the allotted problem statement step by step. It explains the problem, identifies the stakeholders and their challenges, studies the current manual approach, proposes a software solution based on software engineering principles, justifies the chosen development methodology, and finally discusses the expected outcomes, risks, limitations, and future enhancements. The Jira Kanban board prepared for the project is included as practical evidence.

3. PROBLEM STATEMENT

Fleet operators may manage many vehicles, each with different service dates, usage levels, faults, and repair requirements. If these details are kept manually in notebooks or separate files, information may be missed or become difficult to find. A vehicle may not receive servicing on time, a reported fault may remain unresolved, or repair expenses may not be tracked properly.

The proposed Intelligent Vehicle Maintenance Management System aims to keep vehicle maintenance information organized and help staff plan service activities. It is intended to support timely maintenance reminders, fault tracking, repair-cost monitoring, and fleet maintenance reporting.

3.1 Understanding the Problem in Simple Words

In simple terms, the problem is a lack of one reliable place where everything about a vehicle can be found. A fleet may contain cars, buses, trucks, and vans, and every vehicle needs oil changes, inspections, tyre checks, and repairs at different times. When this information is scattered across notebooks, spreadsheets, and memory, the staff cannot clearly see which vehicle is due for service, which fault is still open, or how much money has already been spent on repairs.

The result is a chain of avoidable troubles. A missed service leads to a breakdown, the breakdown delays the vehicle's work, the emergency repair costs more than planned maintenance would have, and the manager has no accurate figures to understand why costs keep rising. The core problem is therefore not the vehicles themselves but the poor organization, tracking, and visibility of maintenance information.

3.2 Need for the System

- To keep vehicle information in an organized place.
- To avoid missing scheduled maintenance.
- To record vehicle faults and follow their repair status.

- To maintain service and repair history for each vehicle.
- To track spare parts and repair expenses.
- To understand vehicle usage and plan servicing more effectively.

3.3 Objectives of the Proposed System

- Maintain vehicle records such as registration number, type, model, and current status.
- Track service history, including service date, work performed, and repair details.
- Schedule preventive maintenance based on time or vehicle usage.
- Record faults, their severity, and whether the repair is pending or completed.
- Generate reminders for upcoming or overdue maintenance.
- Track spare parts used during repair and record repair costs.
- Monitor vehicle utilization, such as distance travelled or operating hours.
- Generate reports that summarize maintenance activities and fleet expenses.

3.4 Scope of the Problem

The scope of the system covers the complete maintenance life of every vehicle in a fleet, from registering the vehicle to reviewing management reports. It includes vehicle records, service history, preventive maintenance planning, fault management, reminders, spare-parts and repair-cost tracking, vehicle utilization, and fleet reports.

The practical work in this report is limited to using Jira to organize the project activities. A Jira space named “Intelligent Vehicle Maintenance Management System” has been created with a Kanban board. Work items are represented as cards and arranged in To Do, In Progress, and Done columns. This board tracks the development work; it is not the completed vehicle maintenance software itself.

4. KEY STAKEHOLDERS AND CHALLENGES

Several people are affected by, or responsible for, the maintenance of a fleet. Identifying them early ensures that the software meets the needs of everyone who will use it or depend on it.

Stakeholder	Roles and Responsibilities	Major Challenges
Fleet Manager	Plans maintenance, approves repairs and budgets, reviews fleet reports, and ensures vehicles are available for use.	No accurate view of fleet condition; cannot compare costs; decisions are based on incomplete records.
Maintenance Staff / Technicians	Perform servicing and repairs, update service history, record spare parts used and work completed.	Missing fault details, repeated paperwork, and unclear priority of pending jobs.
Drivers / Vehicle Operators	Use the vehicles daily, report faults, and provide distance or usage readings.	No easy way to report problems; faults may be forgotten or reported late.
Administrative / Accounts Staff	Maintain vehicle registration details, track repair bills, and monitor expenses.	Repair costs and spare-part bills are scattered and hard to total.
Spare-Parts Suppliers / Service Centres	Supply parts and perform external repairs.	Unclear demand for parts; delays when requirements are not planned.
Project Team (Developers, Tester, Scrum/ Project Lead)	Design, develop, test, and deliver the system and manage tasks on the Jira board.	Changing requirements, task tracking, and meeting deadlines.
Faculty Guide / Evaluator	Reviews the project and the Jira practical for academic assessment.	Needs clear evidence of planning and progress.

4.1 Major Challenges Faced

- **Missed maintenance:** Service dates are remembered informally, so servicing is delayed or skipped.

- **Unresolved faults:** Faults are noted on paper or verbally and may never be followed up.
- **Untracked costs:** Repair and spare-part expenses cannot be totalled correctly for each vehicle.
- **Scattered records:** Information is stored in different notebooks and files, making searching slow.
- **Lack of reporting:** Managers cannot easily see vehicle usage, condition, and expenses together.

5. ANALYSIS OF THE CURRENT APPROACH

5.1 Existing System or Current Process

In many small and medium fleets, maintenance is handled manually. Vehicle details are written in notebooks or register books, service dates are noted on paper or in separate spreadsheet files, and faults are reported verbally to the maintenance staff. Repair bills are collected in files, and the manager prepares expense summaries by hand when required. Reminders depend entirely on the memory of individual staff members.

5.2 Strengths of the Current Approach

- It is simple and needs no special training or technology.
- It has almost no initial cost, so it is easy to start with a small number of vehicles.
- It is flexible, since notes can be written in any format.

5.3 Limitations of the Current Approach

- Information is easily lost, damaged, duplicated, or entered incorrectly.
- Searching the history of a particular vehicle takes a long time.
- There are no automatic reminders, so upcoming or overdue maintenance is easily missed.

- Fault status (pending or completed) cannot be tracked reliably.
- Repair costs and spare-parts usage are difficult to calculate and compare.
- The method does not scale as the number of vehicles grows.
- Reports for management take a lot of manual effort and may be inaccurate.

5.4 Gaps to Be Addressed

The analysis shows clear gaps: there is no centralized record of vehicles, no scheduling based on time or usage, no structured fault tracking, no automated reminders, no cost and parts monitoring, and no reporting. The proposed system is designed to fill each of these gaps.

6. PROPOSED SOLUTION USING SOFTWARE ENGINEERING PRINCIPLES

The proposed solution is a web-based Intelligent Vehicle Maintenance Management System in which staff can store vehicle data, plan servicing, record faults, receive reminders, and view reports from a single application backed by a central database.

6.1 Major Functional Modules

Module	Description
Vehicle Records	Stores basic details for each vehicle, such as vehicle number, type, model, and status.
Service History	Maintains previous service dates, repairs, and notes about work completed.
Preventive Maintenance	Helps plan routine service before a fault becomes serious.
Fault Management	Records vehicle faults and follows them until they are resolved.
Maintenance Reminders	Identifies upcoming or overdue service based on date or usage.
Spare Parts and Repair Costs	Records parts used and expenses for each repair.

Vehicle Utilization	Tracks distance or operating time to support service planning.
Fleet Reports	Summarizes vehicle servicing, faults, costs, and maintenance status.

6.2 Proposed Workflow

The application follows a simple workflow. First, the staff member adds the vehicle details. The service history is then recorded whenever maintenance is performed. The system checks service dates or usage information to identify upcoming maintenance. If a fault is reported, it is recorded and assigned for repair. The repair details, spare parts, and costs are updated after the work is completed. Finally, fleet reports can be reviewed by the manager.

1. Add vehicle details.
2. Record service and usage information.
3. Check the next maintenance due date.
4. Record faults and repair requirements.
5. Send or display maintenance reminders.
6. Update spare-parts usage and repair costs.
7. Review fleet maintenance reports.

6.3 Functional Requirements

- Add, edit, view, and deactivate vehicle records.
- Record and display complete service and repair history for each vehicle.
- Schedule preventive maintenance by date, distance, or operating hours.
- Log faults with severity and status (pending, in repair, completed).
- Generate and display reminders for upcoming and overdue maintenance.
- Record spare parts used and calculate repair costs.

- Record vehicle utilization readings.
- Provide login with role-based access and generate fleet reports.

6.4 Non-Functional Requirements

- **Performance:** The system should respond to normal requests within a few seconds.
- **Reliability:** Data must be stored safely with regular backups and consistent availability.
- **Usability:** The interface should be simple enough for non-technical staff.
- **Security:** Only authorized users may view or change data.
- **Scalability:** The system should support growth in vehicles and users.
- **Maintainability:** The code should be easy to understand, correct, and extend.

6.5 Software Engineering Principles Applied

Principle	How It Is Incorporated
Modularity	Each function (vehicles, services, faults, reminders, costs, reports) is a separate module with clear interfaces, so modules can be built and tested independently.
Scalability	A central database and a layered architecture allow more vehicles, users, and branches to be added without redesign.
Maintainability	Clean, documented code, separation of presentation, business logic, and data layers, and version control make changes and bug fixes easy.
Reliability	Input validation, error handling, data backups, and testing reduce failures and prevent data loss.
Usability	Simple forms, dashboards, colour-coded status, and clear reminders help staff use the system with minimal training.
Security	Role-based access, password protection, and encrypted data transfer protect sensitive maintenance and cost information.

7. JUSTIFICATION OF THE CHOSEN METHODOLOGY (AGILE – KANBAN)

The Agile approach with a Kanban board is selected as the software development framework for this project. Kanban is a lean, visual method in which each piece of work is a card that moves across columns showing its progress.

7.1 Why Agile Kanban Is Suitable

- **Visual workflow:** Every module of the system appears as a card, so the whole team can see what is pending, in progress, and completed.
- **Flexibility:** Requirements such as new report types or reminder rules may change; cards can be added or reprioritized at any time.
- **Focus:** Limiting the work in progress prevents the team from starting too many modules at once.
- **Continuous delivery:** Modules can be delivered one after another instead of waiting for the entire system.
- **Simplicity:** It requires little ceremony and is ideal for a small team and a short academic project.

Waterfall was not chosen because it needs all requirements fixed in advance and gives feedback only at the end. Scrum is also suitable but requires fixed sprints and several formal meetings, which are more than this small project needs. Kanban gives the right balance of structure and flexibility.

7.2 Support for Development and Maintenance

Kanban supports successful development by making progress transparent and by exposing bottlenecks early. After release, the same board can be used to manage maintenance work such as bug fixes and enhancements, so the team follows one consistent process throughout the system’s life.

8. JIRA IMPLEMENTATION: STEP-BY-STEP

Jira helps the project team organize development tasks, see their current status, and understand what remains to be done. The Kanban board provides a visual workflow. It does not automatically monitor vehicles or generate real maintenance alerts unless such features are implemented in the actual application.

1. **Open Jira:** Sign in to Jira and open the project or space area.
2. **Create a space:** Create a space named Intelligent Vehicle Maintenance Management System.
3. **Select a template:** Choose the Kanban template to use a visual task board.
4. **Open the board:** Open the Board tab and check the To Do, In Progress, and Done columns.
5. **Add tasks:** Create work items for the major objectives of the proposed system.
6. **Arrange tasks:** Place each work item in the column that matches its current progress.
7. **Update progress:** Move a card to In Progress when work begins and to Done only after completion.
8. **Review and capture:** Check the board and take a screenshot as evidence for the practical report.

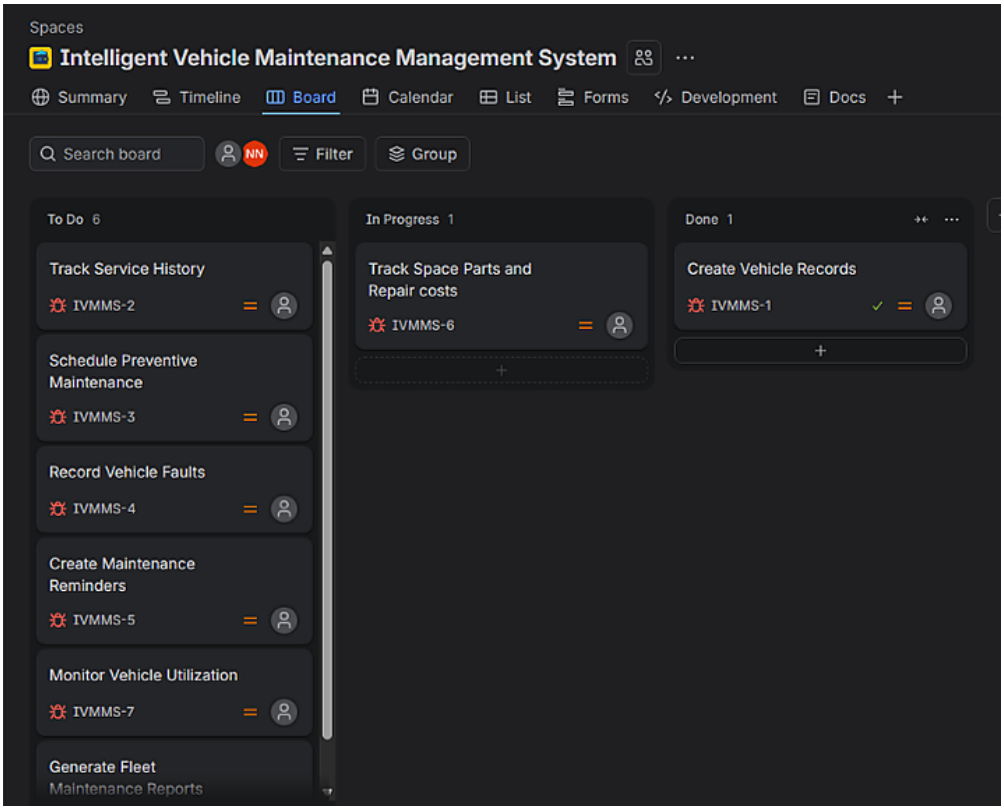


Figure 1: Jira Kanban board of the Intelligent Vehicle Maintenance Management System

8.1 Kanban Board Columns

- **To Do:** Tasks that have not started.
- **In Progress:** Tasks currently being worked on.
- **Done:** Tasks that have been completed.

In the board shown above, the task “Create Vehicle Records” (IVMMS-1) is in the Done column, “Track Spare Parts and Repair Costs” (IVMMS-6) is In Progress, and the remaining tasks, namely Track Service History, Schedule Preventive Maintenance, Record Vehicle Faults, Create Maintenance Reminders, Monitor Vehicle Utilization, and Generate Fleet Maintenance Reports, are in To Do.

9. EXPECTED OUTCOMES AND LIMITATIONS

9.1 Expected Benefits

- More organized vehicle records and service histories.
- Better planning of preventive maintenance.
- Faster identification and tracking of faults.
- Improved awareness of repair costs and spare-parts usage.
- Reduced chance of unexpected breakdowns.
- Improved vehicle safety and operational reliability.

9.2 How the Solution Addresses the Challenges

Centralized vehicle records remove scattered information; preventive scheduling and reminders prevent missed service; fault management ensures every reported problem is followed until it is resolved; cost and parts tracking gives accurate expense figures; and fleet reports give managers the visibility they need for informed decisions.

9.3 Risks, Implementation Challenges, and Limitations

- Staff may resist moving from manual methods and will need basic training.
- Incorrect or late data entry will reduce the accuracy of reminders and reports.
- Changing requirements may affect the schedule and effort.
- Real-time monitoring is not possible without extra hardware such as GPS or sensors.
- The Jira board only manages the project tasks; it does not run maintenance alerts.
- Security and data-backup measures must be planned from the beginning.

9.4 Future Enhancements

- Integration with GPS and on-board diagnostic (OBD) devices for automatic distance and fault readings.
- A mobile application for drivers to report faults instantly.

- SMS, e-mail, and app notifications for maintenance reminders.
- Predictive maintenance using machine learning to forecast failures.
- Fuel monitoring, driver performance analysis, and advanced cost dashboards.

10. CONCLUSION

The Intelligent Vehicle Maintenance Management System addresses a real problem faced by fleet operators: scattered, manual maintenance information that leads to missed service, unresolved faults, and untracked costs. By organizing vehicle records, scheduling preventive maintenance, tracking faults, parts, and costs, and generating reports, the system improves safety, reliability, and decision-making. The use of Jira with a Kanban board demonstrates how Agile project management can organize and monitor the development tasks, and this practical exercise achieves the aim of creating and tracking the project through To Do, In Progress, and Done stages.